Guía de implementación: proyectos Spring Boot con Docker

Oct 26, 2026 · @fagarra

Propósito y prerrequisitos

Al terminar esta guía, el estudiante tendrá un proyecto Spring Boot empaquetado en una imagen Docker y levantado junto a su base de datos con un solo comando: `docker compose up -d --build`. Es material de apoyo para la Semana 15 de Electiva I y para el Taller 4.

La guía avanza de lo simple a lo completo: primero un servicio solo, luego el servicio con PostgreSQL y por último varios microservicios con Eureka y Gateway. Los fundamentos de Docker (imagen, contenedor, capa, volumen, red) se dan por conocidos desde Desarrollo de Software II.

Herramienta	Versión	Verificación
JDK	21 LTS	`java -version`
Spring Boot	3.5.x	`pom.xml`
Maven	3.9.x (o el wrapper `./mvnw`)	`./mvnw -v`
Docker Desktop / Engine	Última estable	`docker version`
Docker Compose	v2 (incluido en Docker Desktop)	`docker compose version`
Postman	Última estable	Pruebas de endpoints

Recomendación de hardware: 8 GB de RAM para los pasos 1 a 4 y 16 GB para el paso 5 con todo el ecosistema.

Paso 1: preparar el proyecto Spring Boot

El proyecto debe arrancar en local antes de pensar en Docker: un contenedor no arregla una aplicación que no funciona. Se usa `producto-service` del e-commerce como ejemplo.

1. Crear el proyecto en start.spring.io con Java 21, Spring Boot 3.5.x y las dependencias Spring Web, Spring Data JPA, PostgreSQL Driver y Spring Boot Actuator.

2. Externalizar en `application.yml` todo lo que cambia entre entornos, usando placeholders con valor por defecto:

```
server:
  port: 8081
spring:
  application:
    name: producto-service
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/productos}
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:postgres}
  jpa:
    hibernate:
      ddl-auto: update
management:
  endpoints:
    web:
      exposure:
        include: health,info
```

3. Compilar y probar en local: `./mvnw clean package` debe generar `target/producto-service-0.0.1-SNAPSHOT.jar`, y `GET http://localhost:8081/actuator/health` debe responder `{"status":"UP"}`.

Actuator es obligatorio en esta guía: Docker Compose usará `/actuator/health` para saber cuándo el servicio está listo.

Paso 2: escribir el Dockerfile

Cada microservicio lleva su propio `Dockerfile` en la raíz de su carpeta, junto al `pom.xml`. Se presentan dos versiones: una mínima para entender la idea y una multi-stage, que es la exigida en el Taller 4.

Versión mínima (solo para entender)

Requiere compilar antes en local con `./mvnw clean package`.

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Sus límites: depende del Maven y el JDK de cada equipo, corre como root y cualquier cambio de código vuelve a copiar todas las dependencias.

Versión multi-stage (recomendada)

Compila dentro de Docker, separa el JAR en capas y ejecuta con un usuario sin privilegios.

```
# ---- Etapa 1: compilar ----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# ---- Etapa 2: extraer capas del JAR ----
FROM eclipse-temurin:21-jre-alpine AS layers
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
RUN java -Djarmode=tools -jar app.jar extract --layers --destination
extracted

# ---- Etapa 3: imagen final ----
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
USER spring
WORKDIR /app
COPY --from=layers /app/extracted/dependencies/ ./
COPY --from=layers /app/extracted/spring-boot-loader/ ./
COPY --from=layers /app/extracted/snapshot-dependencies/ ./
COPY --from=layers /app/extracted/application/ ./
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Etapa	Qué hace	Por qué importa
1. Compilar	Descarga dependencias y genera el JAR	No depende de lo instalado en el equipo; `go-offline` cachea dependencias mientras el `pom.xml` no cambie
2. Extraer	Divide el JAR en capas (Spring Boot 3.3+)	Prepara la reutilización de caché

Etapa	Qué hace	Por qué importa
3. Ejecutar	Copia las capas de la más estable a la más cambiante	Un cambio de código solo reconstruye la capa `application/`; la imagen final no lleva JDK ni Maven

Archivo .dockerignore

Evita enviar al build archivos innecesarios o secretos.

```
target/
.git/
.idea/
.vscode/
*.iml
.env
```

Si el proyecto es multi-módulo con un `pom.xml` padre (Semana 3), el build debe ejecutarse con el contexto en la raíz y `mvn -pl producto-service -am package`.

Paso 3: construir y ejecutar la imagen

Con el Dockerfile listo, la imagen se construye y se prueba sola antes de pasar a Compose. Los comandos se ejecutan desde la carpeta del servicio.

1. Construir la imagen con nombre y versión:

```
docker build -t ecommerce/producto-service:1.0 .
```

2. Verificar la imagen y sus capas:

```
docker images ecommerce/*
docker history ecommerce/producto-service:1.0
```

3. Ejecutar el contenedor publicando el puerto y pasando la configuración por variables de entorno:

```
docker run --rm --name producto -p 8081:8081 \
  -e DB_URL=jdbc:postgresql://host.docker.internal:5432/productos \
  -e DB_USER=postgres \
  -e DB_PASSWORD=postgres \
  ecommerce/producto-service:1.0
```

4. Probar desde Postman: `GET http://localhost:8081/actuator/health`.

Dentro de un contenedor, `localhost` es el propio contenedor, no el equipo. Por eso el ejemplo usa `host.docker.internal` para llegar a un PostgreSQL instalado en la máquina; en Linux hay que añadir `--add-host=host.docker.internal:host-gateway`. El paso 4 elimina esta dependencia al llevar la base de datos a Compose.

Spring Boot traduce las variables de entorno a propiedades (relaxed binding): `SPRING_DATASOURCE_URL` equivale a `spring.datasource.url`. En esta guía se prefieren placeholders explícitos como `DB_URL` porque son más fáciles de leer y documentar.

Paso 4: orquestar servicio y base de datos con Docker Compose

Docker Compose describe en un archivo todos los contenedores, su red, sus volúmenes y su orden de arranque. Dentro de la red que crea Compose, cada servicio se alcanza por su nombre: `postgres-productos` es el hostname de la base de datos.

Estructura mínima del proyecto:

```
producto-service/
├── docker-compose.yml
├── .env            (no se versiona)
├── .env.example    (sí se versiona)
├── .dockerignore
├── Dockerfile
├── pom.xml
└── src/
```

Archivo `.env` con las credenciales (copiar desde `.env.example` y nunca subirlo al repositorio):

```
DB_USER=ecommerce
DB_PASSWORD=cambiar_esta_clave
```

Archivo `docker-compose.yml`:

```
services:
  postgres-productos:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: productos
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pg-productos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
```

```
      interval: 10s
      retries: 5

  producto-service:
    build: .
    image: ecommerce/producto-service:1.0
    ports:
      - "8081:8081"
    environment:
      DB_URL: jdbc:postgresql://postgres-productos:5432/productos
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      JAVA_TOOL_OPTIONS: -XX:MaxRAMPercentage=75
    depends_on:
      postgres-productos:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:8081/actuator/health"]
      interval: 10s
      retries: 10
    restart: on-failure

volumes:
  pg-productos:
```

Elemento	Para qué sirve
`build: .`	Construye la imagen con el Dockerfile del paso 2
`volumes`	Conserva los datos de PostgreSQL aunque se elimine el contenedor
`healthcheck`	Define cuándo un contenedor está realmente listo, no solo encendido
`depends_on` + `service_healthy`	El servicio espera a que PostgreSQL responda antes de arrancar
`restart: on-failure`	Reinicia el servicio si falla al arrancar
`JAVA_TOOL_OPTIONS`	Limita el heap de la JVM al 75 % de la memoria del contenedor

Levantar todo y verificar:

```
docker compose up -d --build
docker compose ps
```

Cuando ambos contenedores muestren `healthy`, `GET http://localhost:8081/actuator/health` debe responder `UP` con la base de datos incluida.

El healthcheck usa `wget` porque viene en Alpine; `curl` no está instalado por defecto en esa imagen.

Paso 5: llevar varios microservicios a Compose

Con más de un servicio, el `docker-compose.yml` pasa a la raíz del repositorio y cada servicio conserva su propio Dockerfile. Solo el Gateway y los paneles publican puertos al equipo; el resto queda dentro de la red de Compose.

```
ecommerce/
├── docker-compose.yml
├── .env / .env.example
├── config-server/     Dockerfile
├── eureka-server/     Dockerfile
├── api-gateway/       Dockerfile
├── producto-service/  Dockerfile
└── pedido-service/    Dockerfile
```

Servicio	Puerto interno	Publicado al equipo	Depende de
postgres-productos / postgres-pedidos	5432	No	—
config-server	8888	No	—
eureka-server	8761	Sí (panel)	config-server
producto-service	8081	No	eureka-server, postgres-productos
pedido-service	8082	No	eureka-server, postgres-pedidos
api-gateway	8080	Sí (entrada única)	eureka-server

Ajustar la configuración de cada servicio

Las URLs de Config Server y Eureka se externalizan con placeholders, de modo que el mismo JAR funciona en local y en Docker:

```
spring:
  config:
    import: optional:configserver:${CONFIG_SERVER_URL:http://localhost:8888}
eureka:
  client:
    service-url:
      defaultZone: ${EUREKA_URL:http://localhost:8761/eureka/}
  instance:
    prefer-ip-address: true
```

Se usa un placeholder para `defaultZone` porque las claves de mapa no se traducen de forma fiable desde variables como `EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`. Config Server y Eureka Server también necesitan Actuator para su healthcheck.

Fragmento del docker-compose.yml

Un bloque `x-spring-env` con un anchor YAML evita repetir las variables comunes en cada servicio:

```
x-spring-env: &spring-env
  CONFIG_SERVER_URL: http://config-server:8888
  EUREKA_URL: http://eureka-server:8761/eureka/
  JAVA_TOOL_OPTIONS: -XX:MaxRAMPercentage=75

services:
  config-server:
    build: ./config-server
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:8888/actuator/health"]
      interval: 10s
      retries: 10

  eureka-server:
    build: ./eureka-server
    ports: ["8761:8761"]
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
    depends_on:
      config-server:
        condition: service_healthy
    healthcheck:
```

```
      test: ["CMD", "wget", "-qO-", "http://localhost:8761/actuator/health"]
      interval: 10s
      retries: 10

  producto-service:
    build: ./producto-service
    deploy:
      replicas: 2
      resources:
        limits:
          memory: 512M
    environment:
      <<: *spring-env
      DB_URL: jdbc:postgresql://postgres-productos:5432/productos
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
    depends_on:
      eureka-server:
        condition: service_healthy
      postgres-productos:
        condition: service_healthy

  api-gateway:
    build: ./api-gateway
    ports: ["8080:8080"]
    environment: *spring-env
    depends_on:
      eureka-server:
        condition: service_healthy

  # postgres-productos, postgres-pedidos y pedido-service siguen el patrón
del paso 4
```

Con dos réplicas, `producto-service` no publica puertos: dos contenedores no pueden ocupar el mismo puerto del equipo. Ambas réplicas aparecen en el panel de Eureka (`http://localhost:8761`) y el Gateway reparte las peticiones entre ellas. En equipos de 8 GB conviene dejar `replicas: 1`.

`depends_on` solo ordena el arranque. Si un servicio se reinicia después, los patrones de resiliencia de la Semana 9 y `restart: on-failure` siguen siendo necesarios.

Comandos y errores frecuentes

Estos comandos cubren casi todo el trabajo diario con Compose y se ejecutan desde la carpeta donde está `docker-compose.yml`.

Comando	Para qué sirve
`docker compose up -d --build`	Construye las imágenes y arranca todo en segundo plano
`docker compose ps`	Muestra estado y salud (`healthy` / `unhealthy`) de cada contenedor
`docker compose logs -f producto-service`	Sigue los logs de un servicio
`docker compose up -d --build producto-service`	Reconstruye y reinicia solo un servicio
`docker compose stop producto-service`	Detiene un servicio (útil para fallas controladas)
`docker compose exec producto-service sh`	Abre una terminal dentro del contenedor
`docker compose config`	Valida el YAML y muestra las variables ya resueltas
`docker stats`	Muestra consumo real de CPU y memoria por contenedor
`docker compose down`	Detiene y elimina contenedores y red; conserva los volúmenes
`docker compose down -v`	Igual que el anterior, pero también borra los volúmenes y sus datos

Síntoma	Causa probable	Solución
`Connection refused` a `localhost`	Dentro del contenedor, `localhost` es el propio contenedor	Usar el nombre del servicio de Compose
`depends_on` no espera	Falta `healthcheck` o `condition: service_healthy`	Agregar ambos; verificar que el servicio tenga Actuator
`port is already allocated`	El servicio sigue corriendo en local, o un servicio escalado publica puerto	Detener el proceso local; no publicar puertos en servicios con réplicas

Síntoma	Causa probable	Solución
`Exited (137)` o reinicios en bucle	El contenedor se quedó sin memoria	Límite de memoria + `MaxRAMPercentage`; revisar `docker stats`
Los cambios de código no aparecen	Se reutilizó la imagen anterior	`docker compose up -d --build`
Desaparecieron los datos	Se usó `down -v` o no había volumen con nombre	Declarar el volumen; usar `down` sin `-v`
El servicio aparece como UNKNOWN en Eureka	Falta `spring.application.name`	Definirlo en `application.yml`
El healthcheck falla siempre	`curl` no existe en la imagen Alpine	Usar `wget -qO-`

Lista de chequeo y entregable

El trabajo está completo cuando otra persona clona el repositorio en un equipo limpio con Docker y levanta todo con `docker compose up -d --build`, sin pasos adicionales.

- [ ] Cada servicio tiene un Dockerfile multi-stage con imagen `eclipse-temurin:21-jre-alpine` y tag fijo (nunca `latest`)
- [ ] Cada servicio tiene `.dockerignore` y corre con un usuario no root
- [ ] URLs, credenciales y hosts se leen de variables de entorno con valores por defecto
- [ ] Todos los servicios exponen `/actuator/health` y tienen `healthcheck` en Compose
- [ ] El orden de arranque se resuelve con `depends_on` y `condition: service_healthy`
- [ ] PostgreSQL usa un volumen con nombre
- [ ] Las credenciales están en `.env`, que no se versiona; el repositorio incluye `.env.example`
- [ ] Solo el Gateway y los paneles publican puertos al equipo
- [ ] `docker compose ps` muestra todos los contenedores en `healthy`

El README del repositorio debe incluir: diagrama de arquitectura, requisitos, instrucciones de despliegue, ejemplos de uso (colección Postman) y una captura de `docker compose ps` con todo en `healthy`.

Lectura de referencia: Docker Compose overview, documentación oficial de Docker.